CardSense Deep Learning Project Report

Group Members

Name	CMS ID
Abdul Wahab	464018
Rana Muhammad Hammad Arif	475203
AbuBakr Amjad	455037

Department of Computing

Deep Learning Project Documentation

1. Project Vision, Purpose, and Real-World Motivation

CardSense is designed as a practical computer-vision and decision-support system for live card gameplay, not as a toy detector that works only on clean benchmark images. The central idea is to help a player maintain reliable situational awareness while cards are moving quickly, hands overlap, lighting changes, and human attention is divided. In real play, people lose track of what has already appeared, what remains probable, and which action is most defensible under uncertainty. This project exists to close that gap by combining robust perception, game-state reasoning, and low-latency guidance into one continuously operating system.

A second defining choice is architectural reuse across game types. While the current implementation depth is strongest for blackjack-oriented reasoning, the broader design clearly aims to support multiple card games through swappable rule and recommendation modules. In other words, CardSense is not just one model plus one script. It is a full pipeline where sensing, state reconstruction, and strategy logic are separated well enough to evolve independently.

2. End-to-End Project Scope Across the Repository

This report covers the complete workspace and explains how each major area contributes to the final system behavior.

The top-level files establish project identity and operations. `README.md` defines setup and usage direction, `requirements.txt` and `package.json` control Python and frontend dependencies, `pytest.ini` sets testing behavior, and deployment assets such as `docker-compose.yml`, `docker-compose.prod.yml`, `Dockerfile.backend`, and `Dockerfile.worker` establish reproducible runtime packaging. The report and planning artifacts at the root, including this document, describe design intent and delivery expectations.

The `backend/` tree is the operational intelligence core. It contains ingestion logic, inference coordination, game-state tracking, recommendation generation, resilience controls, storage adapters, and configuration. The `worker/` directory isolates inference execution so heavy vision workloads can scale separately from request handling and state logic. The `frontend/` directory delivers the live user surface, including camera-adjacent interaction and recommendation display behavior. The `data/` directory and dataset-related scripts represent the offline model lifecycle foundation.

The `tests/` tree documents expected behavior and regression protection, spanning gameplay correctness, recommendation consistency, load and integration planning placeholders, and reliability validation. The `scripts/` directory supports operational drills, smoke checks, rollout/rollback workflows, and demonstration tooling. The `ops/` directory holds monitoring and alert definitions that enable production-minded observability. The `java/` subtree indicates a parallel or exploratory implementation path that preserves interoperability options and reflects system-level experimentation beyond a single

language runtime.

3. Problem Framing as a Systems Challenge, Not Only a Model Challenge

Card recognition in controlled conditions is straightforward compared with live table conditions. The true challenge emerges when perception must remain stable while cards are partially hidden, reflections alter texture cues, motion blur distorts edges, and multiple events occur nearly at once. For CardSense, quality is determined by the full chain: if detections are accurate but arrive too late, strategy advice becomes stale; if latency is low but state transitions are inconsistent, recommendations lose trustworthiness.

Because of this, the project should be understood as a real-time ML systems problem. Model quality, throughput, queue behavior, concurrency control, event ordering, and fault recovery all jointly determine whether a user experiences reliable assistance. This framing is why the repository includes not only model code, but also dedicated scaling logic, metrics infrastructure, and operational guardrails.

4. Data Strategy and Dataset Construction

The dataset strategy combines publicly available card imagery with custom, scenario-specific captures from real gameplay contexts. Public data provides broad visual coverage and annotation baselines, while custom data contributes the difficult conditions that usually break naive detectors: glare, nonuniform shadows, overlapping cards, hand occlusion, and varied camera angles. This hybrid design is important because benchmark-only training often overestimates field performance.

Labeling strategy is rank-suit specific rather than generic-card detection, enabling downstream state and strategy reasoning to operate on semantically rich outputs. This class granularity is essential for count-aware logic and action recommendation validity. Data preparation is therefore not merely image cleanup; it is an information-shaping process that decides whether later modules can reason correctly.

The project’s preparation workflow also reflects scale awareness. Distributed preprocessing and experiment management pathways indicate that data handling was treated as a throughput concern, not a one-time manual step. That decision matters because repeated retraining and ablation cycles quickly become bottlenecks when preprocessing is ad hoc.

5. Augmentation Design for Field Robustness

Augmentation in CardSense is targeted rather than decorative. Rotational variation, blur simulation, brightness shifts, and occlusion injection are used to mimic concrete failure modes seen during live play. More specialized transformations, such as synthetic glare overlays and card-aware constraints, are

particularly valuable because glossy card stock and directional lighting produce highly nonstationary highlights that confuse models trained on flat-lit images.

The key methodological point is that augmentation is tied to observed deployment conditions. This is the correct approach for robust performance: train for the environment you actually serve, not the environment that is easiest to label. The project's documented gains from augmentation support the idea that robustness improvements came from deliberate data design, not only from architecture swaps.

6. Model Selection, Training, and Tuning Logic

CardSense uses a detector family tuned for real-time constraints, with smaller variants favored when they preserve responsiveness under full pipeline overhead. This is a rational systems-level compromise. A heavier model can increase raw detection quality but still reduce practical utility if it causes queue buildup or stale recommendations.

Training appears to follow an iterative cycle: initialize a baseline detector, fine-tune on mixed public-plus-custom data, evaluate under realistic conditions, then compare alternatives using consistent metrics. Hyperparameter exploration includes confidence threshold tuning, image-size tradeoff studies, augmentation mix experimentation, and class-balancing considerations. The project also references distributed experiment orchestration and tracked runs, which is a strong sign of reproducibility intent even where full hyperparameter tables are not yet fully documented.

An important strength here is that model selection is not isolated from deployment behavior. Choosing a checkpoint based only on offline accuracy would be incomplete; CardSense instead evaluates quality alongside throughput and latency, which better aligns model choice with user impact.

7. Runtime Pipeline and Service Decomposition

The runtime architecture separates ingestion, queueing, inference, state fusion, persistence, and client updates into distinct responsibilities. This decomposition prevents expensive vision work from blocking frame intake and enables scaling where it is most needed. Frames enter a backend ingress layer, are buffered in stream-based middleware, processed by worker processes, and then passed to state logic that validates event freshness, enforces ordering constraints, and updates game context before recommendations are emitted.

This separation is technically significant. It allows independent tuning of worker count, process count, and reducer behavior under changing load patterns. It also creates clean failure boundaries: when inference slows, ingestion can continue for a bounded period; when a worker fails, redistribution mechanisms can recover progress without collapsing the entire service.

Within the `backend/app/` modules, this design appears as a collection of specialized components: game-state handling, recommendation engines (including blackjack-specific strategy logic), consistency checks, storage interfaces, scaling controls, security controls, and feature toggles. In `worker/worker.py`, compute-heavy inference is isolated so horizontal scaling is straightforward.

8. Concurrency, Consistency, and Fault Recovery

CardSense handles concurrency at multiple layers simultaneously: many incoming camera streams, parallel worker execution, and synchronized updates to shared game state. Without safeguards, these layers can produce duplicate updates, stale transitions, and race conditions. The project addresses this using deduplication, freshness checks, lock-based or serialized state mutation patterns, and idempotent persistence behavior.

Fault recovery is treated as a first-class concern rather than an afterthought. Heartbeat-style liveness checks, stream-consumer redistribution semantics, and buffered processing together allow the system to tolerate worker loss while preserving continuity. This resilience posture is one of the most mature aspects of the project because it recognizes that real deployments fail in partial, messy ways rather than in clean all-or-nothing modes.

9. Performance Profile and Observed Outcomes

The project reports strong practical metrics for a student-scale real-time system, including high detection quality, low end-to-end response delay, and substantial throughput gains when worker replicas increase. The important interpretation is not a single score, but the shape of results: quality remains competitive while scale-out improves processing capacity, indicating that bottlenecks were addressed structurally rather than hidden.

Latency targets are especially important because recommendation value decays quickly over time in live play. The architecture’s ability to keep camera-to-guidance delay near real-time boundaries is therefore central to user trust. Throughput scaling demonstrations further suggest that the distributed design is functioning as intended under increased concurrency.

10. Frontend and Interaction Layer

The frontend is where technical quality becomes user experience. Files in `frontend/`, including gameplay-facing views and stylesheets, provide the visual context for state presentation and recommended actions. The interface must communicate changing information quickly without cognitive overload, which requires clean update cadence, clear hierarchy, and stability under frequent refreshes.

This layer also validates backend and model quality in practice. Even accurate predictions can feel unreliable if the display flickers, lags, or presents ambiguous transitions. CardSense's design intent appears to align system outputs with human decision timing, which is essential for adoption in live scenarios.

11. Storage, Monitoring, and Operational Readiness

Persistence in CardSense supports traceability, replay, and post-run diagnosis. Storing transitions and recommendation history makes it possible to audit behavior and investigate mismatches between perception and advice. This is crucial for model/system debugging because many failures are temporal and cannot be reconstructed from final state snapshots alone.

Operational maturity is reinforced by the `ops/` configuration files, which define metric collection and alerting behavior, and by the `scripts/` directory, which includes smoke checks, rollout and rollback helpers, and disaster-recovery style drills. Together, these components show that the project was engineered with lifecycle thinking: deployment, monitoring, maintenance, and recovery are treated as part of the product, not as external concerns.

12. Testing and Validation Coverage

The `tests/` suite indicates deliberate validation across multiple dimensions. Gameplay correctness tests verify rule and state logic, recommendation tests check decision consistency, and additional plan placeholders indicate intended expansion into contract, load, integration, and security validation. This is the correct trajectory for a system where model outputs influence downstream reasoning.

A notable quality here is that testing spans both algorithmic and systems concerns. Real-time ML projects often test model metrics but neglect pipeline behavior under stress; CardSense explicitly includes reliability and state-authoritativeness checks, which reduces the risk of silent regressions as the codebase evolves.

13. Game Logic Breadth and Modular Growth Path

Although blackjack logic is currently the most detailed execution path, the repository structure and report artifacts indicate a broader multi-game ambition. Modules for additional game logic and recommendation pathways suggest a plug-in style growth model where core perception and infrastructure remain stable while strategy modules vary by game.

This architectural stance is strategically strong. It protects prior engineering investment and enables incremental expansion: once detection and state infrastructure are reliable, new game modes become a controlled extension problem rather than a full rewrite.

14. Security and Responsible Use Considerations

The project includes practical control points such as ingest-rate safeguards, registration/binding checks, and environment-level configuration controls. These elements reduce misuse and improve runtime stability. The documented ethical framing emphasizes assistance based on user-visible information rather than illicit extraction of hidden data, aligning the project with educational and accessibility-oriented use.

Future revisions should continue strengthening this area with formalized authentication and authorization policy descriptions, transport protection details, and explicit retention governance for stored gameplay records. The current foundation is sensible, but policy-level clarity will be important as deployment contexts broaden.

15. Documentation Gaps and What to Add Next

CardSense already demonstrates substantial engineering depth, but publication-grade completeness would require tighter experiment reporting. The highest-impact additions are full training configuration tables, explicit data split methodology, per-class performance breakdowns, augmentation ablation summaries, and deterministic reproduction instructions for best checkpoints.

These additions would not change the core system quality, but they would improve scientific transparency and make independent validation easier for instructors, reviewers, and future contributors.

16. Final Assessment

CardSense succeeds because it treats deep learning as one component of an end-to-end real-time decision system. The repository demonstrates meaningful integration of data engineering, model tuning, distributed execution, fault tolerance, frontend delivery, testing discipline, and operational controls. The result is a technically credible platform that goes beyond isolated model accuracy and addresses the full lifecycle of intelligent assistance in live environments.

Most importantly, every major project area in this workspace contributes to that outcome: backend reasoning and orchestration, worker-scale inference, user-facing frontend behavior, monitoring and alerting, deployment automation, validation suites, and future-facing modular game support. That breadth is what makes this project substantial and academically defensible as a complete deep learning systems effort.